# esde2

July 24, 2025

## 1 Enhanced Symbolic Discovery Engine (ESDE)

A hybrid neural-symbolic system for solving mathematical, logical, and geometric problems, targeting AIME, SAT, and GSM8K benchmarks. Integrates neural fields for spatial reasoning and real-world 3D scenarios.

### 1.1 Features

- Parses natural language into structured sketches.
- Generates and verifies algebraic and geometric rules.
- Evolves rules using contrastive learning.
- Embeds expressions for semantic retrieval and pattern discovery.
- Models 3D geometries with neural fields.
- Stores knowledge for meta-learning.
- Refines outputs for clarity.

### 1.2 Dependencies

- Python 3.8+
- `pydantic`, `sympy`, `z3-solver`, `transformers`, `cma`, `scikit-learn`, `mlxtend`, `torch`, `tinycudann`, `numpy`, `networkx`, `faiss-cpu`, `pytest`, `json_repair`, `redis`

### 1.3 Setup

Run in a Jupyter environment with dependencies installed: “‘bash pip install pydantic sympy z3-solver transformers cma scikit-learn mlxtend torch tinycudann numpy networkx faiss-cpu pytest json_repair redis

[26]:
```
### **Imports and Setup (Code)**

# Import dependencies for ESDE functionality
import json
import re
import traceback
import requests
import numpy as np
from typing import List, Dict, Union
from pydantic import BaseModel, ValidationError
from sympy import sympify, Equality, Add, Mul, Sum, symbols, simplify, Expr
```

```
from sympy.abc import k, n, x, i
from sympy.core.relational import Relational
from sympy.parsing.sympy_parser import parse_expr
from sklearn.cluster import KMeans
from sklearn.metrics.pairwise import euclidean_distances
import networkx as nx
import pandas as pd
from mlxtend.frequent_patterns import fpgrowth
import torch
import faiss
import json_repair
import redis
from concurrent.futures import ThreadPoolExecutor
import asyncio
from z3 import Solver, Int, sat

# Suppress warnings for cleaner output
import warnings
warnings.filterwarnings("ignore")

# Set random seed for reproducibility
np.random.seed(42)
torch.manual_seed(42)

# Ollama wrapper for local Mistral model
def call_ollama(prompt: str, model: str = "mistral", temperature: float = 0.2)␣
 ↪-> str:
    """Call the Ollama API to generate text using a local model."""
    response = requests.post(
        "http://localhost:11434/api/generate",
        json={"model": model, "prompt": prompt, "stream": False, "temperature":␣
 ↪temperature}
    )
    return response.json()["response"].strip()

# Redis cache for LLM outputs
cache = redis.Redis(host='localhost', port=6379, db=0)
```

### 1.4 Multi-Agent Sketch Negotiator

**Purpose**: Parses natural language problem statements into structured JSON sketches, detecting spatial tasks for routing to the geometry module.

**Technical Details**: - Uses Phi-3 (via Ollama) to propose sketches, SymPy/Z3 to validate, and rule-based refinement. - Employs CMA-ES for optimization (simplified here as retries). - Pydantic ensures robust JSON validation.

[27]:
```python
class Sketch(BaseModel):
    """Pydantic model for structured problem sketches."""
    task_type: str
    domain: str
    goal: str
    axioms: List[str]
    method: str

class SketchNegotiator:
    """Negotiates a structured sketch from a natural language problem."""
    def __init__(self, model_name: str = "mistral", max_retries: int = 2):
        self.model_name = model_name
        self.max_retries = max_retries

    def propose_sketch(self, problem: str) -> Sketch:
        """Propose a structured sketch from a problem statement."""
        # Check cache first
        cache_key = f"sketch:{hash(problem)}"
        cached = cache.get(cache_key)
        if cached:
            print("Using cached sketch")
            return Sketch.model_validate_json(cached)

        prompt = f"""You are a proof assistant. Convert the math problem into a␣
↪structured sketch.
        Return a JSON object with:
        - task_type
        - domain
        - goal
        - axioms (list)
        - method (string or list)
        Use only arithmetic/algebraic language. No comments.
        Problem: {problem}"""

        for attempt in range(1, self.max_retries + 1):
            raw = call_ollama(prompt, model=self.model_name)
            print(f"[Attempt {attempt}] LLM raw output:\n{raw}\n")
            try:
                repaired = json_repair.repair_json(raw)
                sketch = self._parse_json(repaired)
                if self._validate_sketch(sketch) and self.
↪_check_logical_consistency(sketch):
                    cache.set(cache_key, sketch.model_dump_json())
                    return sketch
            except Exception as e:
                print(f"Attempt {attempt} failed: {e}")
        return self._fallback_sketch()
```

```
    def _parse_json(self, text: str) -> Sketch:
        """Parse and validate JSON from LLM output."""
        match = re.search(r"\{.*?\}", text, re.DOTALL)
        if not match:
            raise ValueError("No JSON block found")
        block = match.group(0)
        block = re.sub(r',\s*(?=[}\]])', '', block)
        obj = json.loads(block)
        if isinstance(obj.get("method"), list):
            obj["method"] = " \n".join(obj["method"])
        return Sketch.model_validate(obj)

    def _validate_sketch(self, sketch: Sketch) -> bool:
        """Validate sketch for minimum requirements."""
        return bool(sketch.axioms and len(sketch.axioms) >= 2 and sketch.method␣
↪and len(sketch.method) >= 10)

    def _check_logical_consistency(self, sketch: Sketch) -> bool:
        """Check logical consistency of axioms using Z3."""
        solver = Solver()
        for axiom in sketch.axioms:
            try:
                solver.add(sympify(axiom))
            except:
                return False
        return solver.check() == sat

    def _fallback_sketch(self) -> Sketch:
        """Provide a fallback sketch if negotiation fails."""
        return Sketch(
            task_type="proof",
            domain="algebra",
            goal="Prove that the sum of the first n odd numbers is n^2",
            axioms=["An odd number is of the form 2*k - 1", "Sum of first n odd␣
↪numbers = n^2"],
            method="induction"
        )
```

[28]:
```
# Sanity Check for SketchNegotiator
negotiator = SketchNegotiator()
sketch = negotiator.propose_sketch("Prove that the sum of the first n odd␣
 ↪numbers is n squared")
print("Sketch Output:", sketch.model_dump_json(indent=2))
```

```
---------------------------------------------------------------------------
ConnectionRefusedError                    Traceback (most recent call last)
```

```
File ~/.local/lib/python3.10/site-packages/redis/connection.py:385, in␣
 ↪AbstractConnection.connect_check_health(self, check_health)
    384 try:
--> 385     sock = self.retry.call_with_retry(
    386         lambda: self._connect(), lambda error: self.disconnect(error)
    387     )
    388 except socket.timeout:

File ~/.local/lib/python3.10/site-packages/redis/retry.py:87, in Retry.
 ↪call_with_retry(self, do, fail)
     86 try:
---> 87     return do()
     88 except self._supported_errors as error:

File ~/.local/lib/python3.10/site-packages/redis/connection.py:386, in␣
 ↪AbstractConnection.connect_check_health.<locals>.<lambda>()
    384 try:
    385     sock = self.retry.call_with_retry(
--> 386         lambda: self._connect(), lambda error: self.disconnect(error)
    387     )
    388 except socket.timeout:

File ~/.local/lib/python3.10/site-packages/redis/connection.py:797, in␣
 ↪Connection._connect(self)
    796 if err is not None:
--> 797     raise err
    798 raise OSError("socket.getaddrinfo returned an empty list")

File ~/.local/lib/python3.10/site-packages/redis/connection.py:781, in␣
 ↪Connection._connect(self)
    780 # connect
--> 781 sock.connect(socket_address)
    783 # set the socket_timeout now that we're connected

ConnectionRefusedError: [Errno 111] Connection refused

During handling of the above exception, another exception occurred:

ConnectionError                           Traceback (most recent call last)
Cell In[28], line 3
      1 # Sanity Check for SketchNegotiator
      2 negotiator = SketchNegotiator()
----> 3 sketch =␣
 ↪negotiator.propose_sketch("Prove that the sum of the first n odd numbers is n squared")
      4 print("Sketch Output:", sketch.model_dump_json(indent=2))

Cell In[27], line 19, in SketchNegotiator.propose_sketch(self, problem)
     17 # Check cache first
```

```
     18 cache_key = f"sketch:{hash(problem)}"
---> 19 cached = cache.get(cache_key)
     20 if cached:
     21     print("Using cached sketch")

File ~/.local/lib/python3.10/site-packages/redis/commands/core.py:1829, in␣
 ↪BasicKeyCommands.get(self, name)
   1823 def get(self, name: KeyT) -> ResponseT:
   1824     """
   1825     Return the value at key ``name``, or None if the key doesn't exist
   1826
   1827     For more information see https://redis.io/commands/get
   1828     """
-> 1829     return self.execute_command("GET", name, keys=[name])

File ~/.local/lib/python3.10/site-packages/redis/client.py:623, in Redis.
 ↪execute_command(self, *args, **options)
    622 def execute_command(self, *args, **options):
--> 623     return self._execute_command(*args, **options)

File ~/.local/lib/python3.10/site-packages/redis/client.py:629, in Redis.
 ↪_execute_command(self, *args, **options)
    627 pool = self.connection_pool
    628 command_name = args[0]
--> 629 conn = self.connection or pool.get_connection()
    631 if self._single_connection_client:
    632     self.single_connection_lock.acquire()

File ~/.local/lib/python3.10/site-packages/redis/utils.py:191, in␣
 ↪deprecated_args.<locals>.decorator.<locals>.wrapper(*args, **kwargs)
    186     elif arg in provided_args:
    187         warn_deprecated_arg_usage(
    188             arg, func.__name__, reason, version, stacklevel=3
    189         )
--> 191 return func(*args, **kwargs)

File ~/.local/lib/python3.10/site-packages/redis/connection.py:1530, in␣
 ↪ConnectionPool.get_connection(self, command_name, *keys, **options)
   1526     self._in_use_connections.add(connection)
   1528 try:
   1529     # ensure this connection is connected to Redis
-> 1530     connection.connect()
   1531     # connections that the pool provides should be ready to send
   1532     # a command. if not, the connection was either returned to the
   1533     # pool before all data has been read or the socket has been
   1534     # closed. either way, reconnect and verify everything is good.
   1535     try:
```

```
File ~/.local/lib/python3.10/site-packages/redis/connection.py:379, in␣
 ↪AbstractConnection.connect(self)
    377 def connect(self):
    378     "Connects to the Redis server if not already connected"
--> 379     self.connect_check_health(check_health=True)

File ~/.local/lib/python3.10/site-packages/redis/connection.py:391, in␣
 ↪AbstractConnection.connect_check_health(self, check_health)
    389     raise TimeoutError("Timeout connecting to server")
    390 except OSError as e:
--> 391     raise ConnectionError(self._error_message(e))
    393 self._sock = sock
    394 try:

ConnectionError: Error 111 connecting to localhost:6379. Connection refused.
```

### 1.5 Neural-Symbolic Rule Engine

**Purpose**: Generates and executes rules for algebraic and geometric problems, blending neural creativity with symbolic precision.

**Technical Details**: - Uses Phi-3 to propose rules in a DSL, executed by SymPy. - Supports parallel rule generation for efficiency.

[29]:
```python
class RuleEngine:
    """Generates and executes symbolic rules for problem-solving."""
    def __init__(self, model_name: str = "mistral", max_candidates: int = 10,␣
 ↪max_retries: int = 2):
        self.model_name = model_name
        self.max_candidates = max_candidates
        self.max_retries = max_retries
        self.templates = {
            "induction": "Eq(Sum(2*i - 1, (i, 1, n)), n**2)",
            "algebra": "Eq(a + b, c)"
        }

    def propose_rules(self, sketch: Sketch) -> List[str]:
        """Propose candidate rules based on the sketch."""
        prompt = f"""Generate {self.max_candidates} rules using templates:␣
 ↪{self.templates}.
        Task: {sketch.task_type}, Goal: {sketch.goal}"""

        with ThreadPoolExecutor(max_workers=4) as executor:
            futures = [executor.submit(lambda: call_ollama(prompt, self.
 ↪model_name))
                       for _ in range(self.max_retries)]
            responses = [f.result() for f in futures]
```

```
        candidates = [self._extract_candidates(resp) for resp in responses]
        return self._clean_and_flatten([c for sublist in candidates for c in␣
↪sublist])

    def _extract_candidates(self, text: str) -> List[str]:
        """Extract rule expressions from LLM output."""
        return [m.group(0) for ln in text.splitlines() if (m := re.
↪search(r"Eq\(.*\)", ln))]

    def _clean_and_flatten(self, rules: List[str]) -> List[str]:
        """Clean and filter valid rule expressions."""
        out = []
        nested = re.compile(r"Eq\(\s*(.+?)\s*,\s*Eq\(\s*(.+?)\s*,\s*(.+?
↪)\s*\)\s*\)")
        for r in rules:
            r0 = re.sub(r"\s+", " ", r)
            if "[" in r0 or "for " in r0 or "->" in r0: continue
            m = nested.match(r0)
            if m:
                r0 = f"Eq({m.group(1)}, {m.group(3)})"
            if re.match(r"^Eq\([^,]+,\s*[^,]+\)$", r0):
                out.append(r0)
        return out

    def execute_rules(self, rules: List[str], sketch: Sketch) -> List[Dict]:
        """Execute rules and validate their results."""
        steps = []
        for rule in rules:
            try:
                expr = sympify(rule)
                steps.append({"rule": rule, "result": str(expr), "valid": expr.
↪is_Equality})
            except Exception as e:
                steps.append({"rule": rule, "result": str(e), "valid": False})
        return steps
```

[30]:
```
# Sanity Check for RuleEngine
rule_engine = RuleEngine()
sketch = Sketch(task_type="proof", domain="algebra", goal="sum(2*k - 1, (k, 1,␣
↪n)) = n**2", axioms=[], method="induction")
rules = rule_engine.propose_rules(sketch)
print("Generated Rules:", rules)
```

```
Generated Rules: ['Eq(3, 4)']
```

### 1.6 Verifier-as-Loss Oracle

**Purpose**: Computes loss for rules to guide optimization.

**Technical Details**: - Uses SymPy for simplification and Z3 for constraint validation. - Calculates loss as squared differences or violation counts.

[31]:
```
class Verifier:
    """Verifies rules and computes loss for optimization."""
    def score_rules(self, rules: List[str]) -> List[Dict]:
        """Score rules based on validity and loss."""
        results = []
        for rule in rules:
            rec = {"rule": rule, "valid": False, "loss": float('inf')}
            try:
                expr = sympify(self._repair_rule(rule))
                if isinstance(expr, Equality):
                    solver = Solver()
                    n = Int('n')
                    solver.add(expr.lhs == expr.rhs)
                    if solver.check() == sat:
                        rec.update(valid=True, loss=0.0)
                    else:
                        diff = simplify(expr.lhs - expr.rhs)
                        rec.update(loss=float(diff.evalf()**2) if diff.
 ↪is_number else 1.0)
            except Exception as e:
                rec["error"] = str(e)
            results.append(rec)
        return results

    def _repair_rule(self, rule: str) -> str:
        """Repair malformed rule strings."""
        r = rule.replace("==", "=")
        r = re.sub(r"Eq\(\s*(.+?)\s*,\s*Eq\(\s*(.+?)\s*,\s*(.+?)\s*\)\s*\)",␣
 ↪r"Eq(\1,\3)", r)
        r = re.sub(r"//.*", "", r)
        return r.strip()
```

[32]:
```
# Sanity Check for Verifier
verifier = Verifier()
test_rules = ["Eq(Sum(2*i - 1, (i, 1, n)), n**2)", "Eq(Sum(2*i - 1, (i, 1, n)),␣
 ↪n**2 + 1)"]
scores = verifier.score_rules(test_rules)
print("Rule Scores:", scores)
```

```
Rule Scores: [{'rule': 'Eq(Sum(2*i - 1, (i, 1, n)), n**2)', 'valid': False,
'loss': 0.0}, {'rule': 'Eq(Sum(2*i - 1, (i, 1, n)), n**2 + 1)', 'valid': False,
'loss': 1.0}]
```

### 1.7 Rulial Contrastive Evolver

**Purpose**: Evolves rules by contrasting successful and unsuccessful applications.

**Technical Details**: - Uses verifier scores to guide evolution. - Applies crossover and mutation on symbolic expressions.

[33]:
```
import random

class Evolver:
    """Evolves rules using contrastive learning."""
    def __init__(self, verifier):
        self.verifier = verifier

    def evolve(self, candidates: List[Dict], domain: str) -> List[Dict]:
        """Evolve rules over generations."""
        population = candidates[:]
        for _ in range(3):  # Generations
            new_pop = []
            for _ in range(len(population)):
                p1, p2 = random.sample(population, 2)
                child = self._crossover(p1, p2)
                child = self._mutate(child)
                new_pop.append(child)
            population = new_pop
        return [min(self.verifier.score_rules([c["rule"] for c in population]),
                    key=lambda x: x["loss"])]

    def _crossover(self, p1: Dict, p2: Dict) -> Dict:
        """Combine parts of two rules."""
        lhs = sympify(p1["rule"].split(',')[0][3:])
        rhs = sympify(p2["rule"].split(',')[1][:-1])
        return {"rule": f"Eq({lhs}, {rhs})"}

    def _mutate(self, cand: Dict) -> Dict:
        """Mutate a rule slightly."""
        expr = sympify(cand["rule"])
        rhs = expr.rhs + random.randint(-2, 2)
        return {"rule": f"Eq({expr.lhs}, {rhs})"}
```

[34]:
```
# Sanity Check for Evolver
evolver = Evolver(verifier)
near_misses = [{"rule": "Eq(Sum(2*i - 1, (i, 1, n)), n**2 + 1)", "valid":␣
 ↪False, "loss": 1.0}]
evolved = evolver.evolve(near_misses, domain="algebra")
print("Evolved Rules:", evolved)
```

```
--------------------------------------------------------------------------
```

```
ValueError                                Traceback (most recent call last)
Cell In[34], line 4
      2 evolver = Evolver(verifier)
      3 near_misses = [{"rule": "Eq(Sum(2*i - 1, (i, 1, n)), n**2 + 1)", "valid :
 ↪ False, "loss": 1.0}]
----> 4 evolved = evolver.evolve(near_misses, domain="algebra")
      5 print("Evolved Rules:", evolved)

Cell In[33], line 14, in Evolver.evolve(self, candidates, domain)
     12 new_pop = []
     13 for _ in range(len(population)):
---> 14     p1, p2 = random.sample(population, 2)
     15     child = self._crossover(p1, p2)
     16     child = self._mutate(child)

File /usr/lib/python3.10/random.py:482, in Random.sample(self, population, k,␣
 ↪counts)
    480 randbelow = self._randbelow
    481 if not 0 <= k <= n:
--> 482     raise ValueError("Sample larger than population or is negative")
    483 result = [None] * k
    484 setsize = 21        # size of a small set minus size of an empty list

ValueError: Sample larger than population or is negative
```

### 1.8 Symbolic Embedding and Clustering

**Purpose**: Embeds expressions for semantic retrieval and clustering.

**Technical Details**: - Uses CodeBERT for semantic embeddings and K-means for clustering. - Combines structural (tree depth) and semantic features.

[35]:
```python
from transformers import AutoTokenizer, AutoModel

class SymbolicEmbedding:
    """Embeds and clusters symbolic expressions."""
    def __init__(self):
        self.tokenizer = AutoTokenizer.from_pretrained("microsoft/
 ↪codebert-base")
        self.model = AutoModel.from_pretrained("microsoft/codebert-base")
        self.kmeans = KMeans(n_clusters=5)

    def tree_to_tokens(self, expr: Expr) -> List[str]:
        """Convert expression tree to token list."""
        tokens = []
        if hasattr(expr, "args"):
            for arg in expr.args:
```

```
                tokens.extend(self.tree_to_tokens(arg))
            tokens.append(str(expr.func))
        else:
            tokens.append(str(expr))
        return tokens

    def embed_expression(self, expr: str) -> tuple[float, np.ndarray]:
        """Embed an expression using CodeBERT."""
        inputs = self.tokenizer(expr, return_tensors="pt", truncation=True,␣
↪padding=True)
        with torch.no_grad():
            outputs = self.model(**inputs)
        embedding = outputs.last_hidden_state.mean(dim=1).squeeze().numpy()
        return len(self.tree_to_tokens(sympify(expr))), embedding

    def cluster_expressions(self, exprs: List[str]) -> List[int]:
        """Cluster expressions based on embeddings."""
        embeddings = [self.embed_expression(e)[1] for e in exprs]
        return self.kmeans.fit_predict(embeddings).tolist()
```

[36]:
```
# Sanity Check for Symbolic Embedding
embedder = SymbolicEmbedding()
test_exprs = ["sum(2*k - 1, (k, 1, n)) = n**2", "sum(2*k, (k, 1, n)) = n*(n+1)"]
depths, embeddings = zip(*[embedder.embed_expression(e) for e in test_exprs])
cluster_ids = embedder.cluster_expressions(test_exprs)
print("Cluster Assignments:", cluster_ids)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
ValueError: Error from parse_expr with transformed code: "sum (Integer (2␣
↪)*Symbol ('k' )-Integer (1 ),(Symbol ('k' ),Integer (1 ),Symbol ('n'␣
↪)))=Symbol ('n' )**Integer (2 )"

The above exception was the direct cause of the following exception:

SyntaxError                               Traceback (most recent call last)
File ~/.local/lib/python3.10/site-packages/sympy/core/sympify.py:507, in␣
↪sympify(a, locals, convert_xor, strict, rational, evaluate)
    506     a = a.replace('\n', '')
--> 507     expr =␣
↪parse_expr(a, local_dict=locals, transformations=transformations, evaluate=ev luate)
    508 except (TokenError, SyntaxError) as exc:

File ~/.local/lib/python3.10/site-packages/sympy/parsing/sympy_parser.py:1090,␣
↪in parse_expr(s, local_dict, transformations, global_dict, evaluate)
   1089     local_dict[i] = null
```

```
-> 1090 raise e from ValueError(f"Error from parse_expr with transformed code:␣
 ↪{code!r}")

File ~/.local/lib/python3.10/site-packages/sympy/parsing/sympy_parser.py:1081,␣
 ↪in parse_expr(s, local_dict, transformations, global_dict, evaluate)
   1080 try:
-> 1081     rv = eval_expr(code, local_dict, global_dict)
   1082     # restore neutral definitions for names

File ~/.local/lib/python3.10/site-packages/sympy/parsing/sympy_parser.py:905, i␣
 ↪eval_expr(code, local_dict, global_dict)
    900 """
    901 Evaluate Python code generated by ``stringify_expr``.
    902
    903 Generally, ``parse_expr`` should be used.
    904 """
--> 905 expr = eval(
    906     code, global_dict, local_dict)  # take local objects in preference
    907 return expr

SyntaxError: invalid syntax (<string>, line 1)

During handling of the above exception, another exception occurred:

SympifyError                                Traceback (most recent call last)
Cell In[36], line 4
      2 embedder = SymbolicEmbedding()
      3 test_exprs = ["sum(2*k - 1, (k, 1, n)) = n**2", "sum(2*k, (k, 1, n)) =␣
 ↪n*(n+1)"]
----> 4 depths, embeddings = zip(*[embedder.embed_expression(e) for e in␣
 ↪test_exprs])
      5 cluster_ids = embedder.cluster_expressions(test_exprs)
      6 print("Cluster Assignments:", cluster_ids)

Cell In[36], line 4, in <listcomp>(.0)
      2 embedder = SymbolicEmbedding()
      3 test_exprs = ["sum(2*k - 1, (k, 1, n)) = n**2", "sum(2*k, (k, 1, n)) =␣
 ↪n*(n+1)"]
----> 4 depths, embeddings = zip(*[embedder.embed_expression(e) for e in␣
 ↪test_exprs])
      5 cluster_ids = embedder.cluster_expressions(test_exprs)
      6 print("Cluster Assignments:", cluster_ids)

Cell In[35], line 27, in SymbolicEmbedding.embed_expression(self, expr)
     25     outputs = self.model(**inputs)
     26 embedding = outputs.last_hidden_state.mean(dim=1).squeeze().numpy()
---> 27 return len(self.tree_to_tokens(sympify(expr))), embedding
```

```
File ~/.local/lib/python3.10/site-packages/sympy/core/sympify.py:509, in␣
 ↪sympify(a, locals, convert_xor, strict, rational, evaluate)
    507     expr = parse_expr(a, local_dict=locals,␣
 ↪transformations=transformations, evaluate=evaluate)
    508 except (TokenError, SyntaxError) as exc:
--> 509     raise SympifyError('could not parse %r' % a, exc)
    511 return expr

SympifyError: Sympify of expression 'could not parse 'sum(2*k - 1, (k, 1, n)) =
 ↪n**2'' failed, because of exception being raised:
SyntaxError: invalid syntax (<string>, line 1)
```

### 1.9 Memory Graph

**Purpose**: Stores and retrieves knowledge for meta-learning.

**Technical Details**: - Uses FAISS for fast embedding search and NetworkX for relational storage. - Mines patterns with FP-growth.

[37]:
```python
class MemoryGraph:
    """Stores and retrieves symbolic knowledge."""
    def __init__(self, embedder: SymbolicEmbedding, dim: int = 768):
        self.graph = nx.Graph()
        self.embedder = embedder
        self.index = faiss.IndexHNSWFlat(dim, 32)  # HNSW with 32 neighbors
        self.exprs = []
        self.metadata = []

    def store(self, expr: str, embedding: np.ndarray, metadata: Dict):
        """Store an expression and its metadata."""
        node_id = len(self.graph.nodes)
        self.graph.add_node(node_id, expr=expr, metadata=metadata)
        self.index.add(embedding.reshape(1, -1))
        self.exprs.append(expr)
        self.metadata.append(metadata)

    def retrieve(self, query_expr: str, k: int = 3) -> List[Dict]:
        """Retrieve similar expressions."""
        _, query_vec = self.embedder.embed_expression(query_expr)
        distances, indices = self.index.search(query_vec.reshape(1, -1), k)
        return [{"expr": self.exprs[i], "distance": distances[0][j], "metadata":
 ↪ self.metadata[i]}
                for j, i in enumerate(indices[0])]

    def mine_patterns(self, exprs: List[str]) -> List[str]:
        """Mine frequent patterns from expressions."""
        token_lists = [self.embedder.tree_to_tokens(sympify(e)) for e in exprs]
```

```
        df = pd.DataFrame(token_lists).fillna(0)
        frequent_patterns = fpgrowth(df, min_support=0.5, use_colnames=True)
        return [str(p) for p in frequent_patterns["itemsets"]]
```

[38]:
```
# Sanity Check for MemoryGraph
memory = MemoryGraph(embedder)
for expr in test_exprs:
    _, emb = embedder.embed_expression(expr)
    memory.store(expr, emb, {"sketch": sketch, "tag": "test_expr"})
retrieved = memory.retrieve(test_exprs[0], k=2)
print("Retrieved Neighbors:", retrieved)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
ValueError: Error from parse_expr with transformed code: "sum (Integer (2␣
 ↪)*Symbol ('k' )-Integer (1 ),(Symbol ('k' ),Integer (1 ),Symbol ('n'␣
 ↪)))=Symbol ('n' )**Integer (2 )"

The above exception was the direct cause of the following exception:

SyntaxError                               Traceback (most recent call last)
File ~/.local/lib/python3.10/site-packages/sympy/core/sympify.py:507, in␣
 ↪sympify(a, locals, convert_xor, strict, rational, evaluate)
    506     a = a.replace('\n', '')
--> 507     expr =␣
 ↪parse_expr(a, local_dict=locals, transformations=transformations, evaluate=ev luate)
    508 except (TokenError, SyntaxError) as exc:

File ~/.local/lib/python3.10/site-packages/sympy/parsing/sympy_parser.py:1090,␣
 ↪in parse_expr(s, local_dict, transformations, global_dict, evaluate)
   1089     local_dict[i] = null
-> 1090 raise e from ValueError(f"Error from parse_expr with transformed code:␣
 ↪{code!r}")

File ~/.local/lib/python3.10/site-packages/sympy/parsing/sympy_parser.py:1081,␣
 ↪in parse_expr(s, local_dict, transformations, global_dict, evaluate)
   1080 try:
-> 1081     rv = eval_expr(code, local_dict, global_dict)
   1082     # restore neutral definitions for names

File ~/.local/lib/python3.10/site-packages/sympy/parsing/sympy_parser.py:905, i␣
 ↪eval_expr(code, local_dict, global_dict)
    900 """
    901 Evaluate Python code generated by ``stringify_expr``.
    902
    903 Generally, ``parse_expr`` should be used.
    904 """
```

```
--> 905 expr = eval(
    906     code, global_dict, local_dict)  # take local objects in preference
    907 return expr

SyntaxError: invalid syntax (<string>, line 1)

During handling of the above exception, another exception occurred:

SympifyError                                Traceback (most recent call last)
Cell In[38], line 4
      2 memory = MemoryGraph(embedder)
      3 for expr in test_exprs:
----> 4     _, emb = embedder.embed_expression(expr)
      5     memory.store(expr, emb, {"sketch": sketch, "tag": "test_expr"})
      6 retrieved = memory.retrieve(test_exprs[0], k=2)

Cell In[35], line 27, in SymbolicEmbedding.embed_expression(self, expr)
     25     outputs = self.model(**inputs)
     26 embedding = outputs.last_hidden_state.mean(dim=1).squeeze().numpy()
---> 27 return len(self.tree_to_tokens(sympify(expr))), embedding

File ~/.local/lib/python3.10/site-packages/sympy/core/sympify.py:509, in␣
 ↪sympify(a, locals, convert_xor, strict, rational, evaluate)
    507     expr = parse_expr(a, local_dict=locals,␣
 ↪transformations=transformations, evaluate=evaluate)
    508 except (TokenError, SyntaxError) as exc:
--> 509     raise SympifyError('could not parse %r' % a, exc)
    511 return expr

SympifyError: Sympify of expression 'could not parse 'sum(2*k - 1, (k, 1, n)) =
 ↪n**2'' failed, because of exception being raised:
SyntaxError: invalid syntax (<string>, line 1)
```

### 1.10 Neural Field Geometry Module

**Purpose**: Models 3D geometries and extracts symbolic constraints.

**Technical Details**: - Placeholder for Instant-NGP integration. - Extracts symbolic constraints from problem statements.

[39]:
```python
class NeuralFieldGeometry:
    """Models 3D geometries and extracts constraints."""
    def extract_constraints(self, problem: str) -> List[str]:
        """Extract geometric constraints (placeholder)."""
        if "geometry" in problem.lower() or "3d" in problem.lower():
            return ["distance(p1, p2) = 5", "area(triangle) = 10"]
        return []
```

[40]:
```
# Sanity Check for NeuralFieldGeometry
geometry = NeuralFieldGeometry()
constraints = geometry.extract_constraints("Find the area of a triangle in 3D")
print("Geometric Constraints:", constraints)
```

```
Geometric Constraints: ['distance(p1, p2) = 5', 'area(triangle) = 10']
```

### 1.11 Feedback Loop

**Purpose**: Refines outputs for clarity and triggers improvements.

**Technical Details**: - Evaluates output clarity and suggests enhancements. - Triggers re-processing if quality is low.

[41]:
```
class FeedbackLoop:
    """Refines outputs and provides feedback."""
    def evaluate_output(self, output: Dict) -> tuple[float, List[str]]:
        """Evaluate output clarity."""
        suggestions = []
        if not output.get("steps"): suggestions.append("1. Add detailed steps.")
        if len(output["proof"]) < 10: suggestions.append("2. Expand proof␣
↪explanation.")
        score = 1.0 - (len(suggestions) / 5.0)
        return score, suggestions

    def trigger_improvements(self, output: Dict, loss: float) -> Dict:
        """Trigger refinements based on score and loss."""
        score, suggestions = self.evaluate_output(output)
        if score < 0.9 or loss > 0.0:
            output["annotations"] = suggestions
        return output
```

[42]:
```
# Sanity Check for FeedbackLoop
feedback = FeedbackLoop()
test_output = {
    "proof": "sum(2*k - 1, (k, 1, n)) = n**2",
    "method": "induction",
    "steps": [],
    "status": "partial",
    "annotations": [],
    "cluster_id": 0
}
refined_output = feedback.trigger_improvements(test_output, 0.5)
print("Refined Output:", refined_output)
```

```
Refined Output: {'proof': 'sum(2*k - 1, (k, 1, n)) = n**2', 'method':
'induction', 'steps': [], 'status': 'partial', 'annotations': ['1. Add detailed
steps.'], 'cluster_id': 0}
```

### 1.12 Main ESDE Pipeline

**Purpose**: Orchestrates all ESDE modules to solve problems.

**Technical Details**: - Integrates negotiation, rule generation, verification, evolution, embedding, memory, and feedback. - Uses async for efficiency.

[43]:
```
class ESDE:
    """Main pipeline for the Enhanced Symbolic Discovery Engine."""
    def __init__(self, model_name: str = "mistral"):
        self.negotiator = SketchNegotiator(model_name)
        self.rule_engine = RuleEngine(model_name)
        self.verifier = Verifier()
        self.evolver = Evolver(self.verifier)
        self.embedder = SymbolicEmbedding()
        self.memory = MemoryGraph(self.embedder)
        self.geometry = NeuralFieldGeometry()
        self.feedback = FeedbackLoop()

    async def solve(self, problem: str) -> Dict:
        """Solve a problem using the ESDE pipeline."""
        sketch = await asyncio.to_thread(self.negotiator.propose_sketch,␣
↪problem)
        rules = await asyncio.to_thread(self.rule_engine.propose_rules, sketch)
        steps = self.rule_engine.execute_rules(rules, sketch)
        scores = self.verifier.score_rules(rules)
        loss_scores = [s["loss"] for s in scores]
        if max(loss_scores) >= 0.9:
            steps = await asyncio.to_thread(self.evolver.evolve, steps, sketch.
↪domain)
        valid_exprs = [s["rule"] for s in steps if s["valid"]]
        for expr in valid_exprs:
            _, emb = self.embedder.embed_expression(expr)
            self.memory.store(expr, emb, {"sketch": sketch, "status": "valid"})
        output = {
            "proof": sketch.goal,
            "method": sketch.method,
            "steps": [s["result"] for s in steps if s["valid"]],
            "status": "verified" if any(s["valid"] for s in steps) else␣
↪"partial",
            "annotations": [],
            "cluster_id": 0
        }
        loss = min(loss_scores) if loss_scores else 1.0
        return self.feedback.trigger_improvements(output, loss)
```

[44]:
```
# Sanity Check for ESDE Pipeline
esde = ESDE()
```

```
algebraic_result = await esde.solve("Prove that the sum of the first n odd␣
 ↪numbers is n squared")
print("Algebraic Result:", algebraic_result)
```

```
---------------------------------------------------------------------------
ConnectionRefusedError                    Traceback (most recent call last)
File ~/.local/lib/python3.10/site-packages/redis/connection.py:385, in␣
 ↪AbstractConnection.connect_check_health(self, check_health)
    384 try:
--> 385     sock = self.retry.call_with_retry(
    386         lambda: self._connect(), lambda error: self.disconnect(error)
    387     )
    388 except socket.timeout:

File ~/.local/lib/python3.10/site-packages/redis/retry.py:87, in Retry.
 ↪call_with_retry(self, do, fail)
     86 try:
---> 87     return do()
     88 except self._supported_errors as error:

File ~/.local/lib/python3.10/site-packages/redis/connection.py:386, in␣
 ↪AbstractConnection.connect_check_health.<locals>.<lambda>()
    384 try:
    385     sock = self.retry.call_with_retry(
--> 386         lambda: self._connect(), lambda error: self.disconnect(error)
    387     )
    388 except socket.timeout:

File ~/.local/lib/python3.10/site-packages/redis/connection.py:797, in␣
 ↪Connection._connect(self)
    796 if err is not None:
--> 797     raise err
    798 raise OSError("socket.getaddrinfo returned an empty list")

File ~/.local/lib/python3.10/site-packages/redis/connection.py:781, in␣
 ↪Connection._connect(self)
    780 # connect
--> 781 sock.connect(socket_address)
    783 # set the socket_timeout now that we're connected

ConnectionRefusedError: [Errno 111] Connection refused

During handling of the above exception, another exception occurred:

ConnectionError                           Traceback (most recent call last)
Cell In[44], line 3
      1 # Sanity Check for ESDE Pipeline
```

```
      2 esde = ESDE()
----> 3 algebraic_result = await esde.solve("Prove that the sum of the first n␣
 ↪odd numbers is n squared")
      4 print("Algebraic Result:", algebraic_result)

Cell In[43], line 15, in ESDE.solve(self, problem)
     13 async def solve(self, problem: str) -> Dict:
     14     """Solve a problem using the ESDE pipeline."""
---> 15     sketch = await asyncio.to_thread(self.negotiator.propose_sketch,␣
 ↪problem)
     16     rules = await asyncio.to_thread(self.rule_engine.propose_rules,␣
 ↪sketch)
     17     steps = self.rule_engine.execute_rules(rules, sketch)

File /usr/lib/python3.10/asyncio/threads.py:25, in to_thread(func, *args,␣
 ↪**kwargs)
     23 ctx = contextvars.copy_context()
     24 func_call = functools.partial(ctx.run, func, *args, **kwargs)
---> 25 return await loop.run_in_executor(None, func_call)

File /usr/lib/python3.10/concurrent/futures/thread.py:58, in _WorkItem.run(self
     55     return
     57 try:
---> 58     result = self.fn(*self.args, **self.kwargs)
     59 except BaseException as exc:
     60     self.future.set_exception(exc)

Cell In[27], line 19, in SketchNegotiator.propose_sketch(self, problem)
     17 # Check cache first
     18 cache_key = f"sketch:{hash(problem)}"
---> 19 cached = cache.get(cache_key)
     20 if cached:
     21     print("Using cached sketch")

File ~/.local/lib/python3.10/site-packages/redis/commands/core.py:1829, in␣
 ↪BasicKeyCommands.get(self, name)
   1823 def get(self, name: KeyT) -> ResponseT:
   1824     """
   1825     Return the value at key ``name``, or None if the key doesn't exist
   1826
   1827     For more information see https://redis.io/commands/get
   1828     """
-> 1829     return self.execute_command("GET", name, keys=[name])

File ~/.local/lib/python3.10/site-packages/redis/client.py:623, in Redis.
 ↪execute_command(self, *args, **options)
    622 def execute_command(self, *args, **options):
--> 623     return self._execute_command(*args, **options)
```

```
File ~/.local/lib/python3.10/site-packages/redis/client.py:629, in Redis.
 ↪_execute_command(self, *args, **options)
    627 pool = self.connection_pool
    628 command_name = args[0]
--> 629 conn = self.connection or pool.get_connection()
    631 if self._single_connection_client:
    632     self.single_connection_lock.acquire()

File ~/.local/lib/python3.10/site-packages/redis/utils.py:191, in␣
 ↪deprecated_args.<locals>.decorator.<locals>.wrapper(*args, **kwargs)
    186     elif arg in provided_args:
    187         warn_deprecated_arg_usage(
    188             arg, func.__name__, reason, version, stacklevel=3
    189         )
--> 191 return func(*args, **kwargs)

File ~/.local/lib/python3.10/site-packages/redis/connection.py:1530, in␣
 ↪ConnectionPool.get_connection(self, command_name, *keys, **options)
   1526     self._in_use_connections.add(connection)
   1528 try:
   1529     # ensure this connection is connected to Redis
-> 1530     connection.connect()
   1531     # connections that the pool provides should be ready to send
   1532     # a command. if not, the connection was either returned to the
   1533     # pool before all data has been read or the socket has been
   1534     # closed. either way, reconnect and verify everything is good.
   1535     try:

File ~/.local/lib/python3.10/site-packages/redis/connection.py:379, in␣
 ↪AbstractConnection.connect(self)
    377 def connect(self):
    378     "Connects to the Redis server if not already connected"
--> 379     self.connect_check_health(check_health=True)

File ~/.local/lib/python3.10/site-packages/redis/connection.py:391, in␣
 ↪AbstractConnection.connect_check_health(self, check_health)
    389     raise TimeoutError("Timeout connecting to server")
    390 except OSError as e:
--> 391     raise ConnectionError(self._error_message(e))
    393 self._sock = sock
    394 try:

ConnectionError: Error 111 connecting to localhost:6379. Connection refused.
```

### 1.13 Demonstrations

**Purpose**: Showcases ESDE on sample tasks.

**Tasks**: 1. Algebraic: "Prove that the sum of the first n odd numbers is $n^2$" 2. Geometric: "Find the area of a triangle with vertices (0,0,0), (1,0,0), (0,1,0)"

[45]:
```
# Demonstrations
esde = ESDE()
algebraic_result = await esde.solve("Prove that the sum of the first n odd␣
 ↪numbers is n squared")
geometric_result = await esde.solve("Find the area of a triangle with vertices␣
 ↪(0,0,0), (1,0,0), (0,1,0)")
print("Algebraic Result:", algebraic_result)
print("Geometric Result:", geometric_result)
```

```
---------------------------------------------------------------------------
ConnectionRefusedError                    Traceback (most recent call last)
File ~/.local/lib/python3.10/site-packages/redis/connection.py:385, in␣
 ↪AbstractConnection.connect_check_health(self, check_health)
    384 try:
--> 385     sock = self.retry.call_with_retry(
    386         lambda: self._connect(), lambda error: self.disconnect(error)
    387     )
    388 except socket.timeout:

File ~/.local/lib/python3.10/site-packages/redis/retry.py:87, in Retry.
 ↪call_with_retry(self, do, fail)
     86 try:
---> 87     return do()
     88 except self._supported_errors as error:

File ~/.local/lib/python3.10/site-packages/redis/connection.py:386, in␣
 ↪AbstractConnection.connect_check_health.<locals>.<lambda>()
    384 try:
    385     sock = self.retry.call_with_retry(
--> 386         lambda: self._connect(), lambda error: self.disconnect(error)
    387     )
    388 except socket.timeout:

File ~/.local/lib/python3.10/site-packages/redis/connection.py:797, in␣
 ↪Connection._connect(self)
    796 if err is not None:
--> 797     raise err
    798 raise OSError("socket.getaddrinfo returned an empty list")

File ~/.local/lib/python3.10/site-packages/redis/connection.py:781, in␣
 ↪Connection._connect(self)
```

```
    780 # connect
--> 781 sock.connect(socket_address)
    783 # set the socket_timeout now that we're connected

ConnectionRefusedError: [Errno 111] Connection refused

During handling of the above exception, another exception occurred:

ConnectionError                           Traceback (most recent call last)
Cell In[45], line 3
      1 # Demonstrations
      2 esde = ESDE()
----> 3 algebraic_result = await esde.solve("Prove that the sum of the first n␣
 ↪odd numbers is n squared")
      4 geometric_result = await esde.solve("Find the area of a triangle with␣
 ↪vertices (0,0,0), (1,0,0), (0,1,0)")
      5 print("Algebraic Result:", algebraic_result)

Cell In[43], line 15, in ESDE.solve(self, problem)
     13 async def solve(self, problem: str) -> Dict:
     14     """Solve a problem using the ESDE pipeline."""
---> 15     sketch = await asyncio.to_thread(self.negotiator.propose_sketch,␣
 ↪problem)
     16     rules = await asyncio.to_thread(self.rule_engine.propose_rules,␣
 ↪sketch)
     17     steps = self.rule_engine.execute_rules(rules, sketch)

File /usr/lib/python3.10/asyncio/threads.py:25, in to_thread(func, *args,␣
 ↪**kwargs)
     23 ctx = contextvars.copy_context()
     24 func_call = functools.partial(ctx.run, func, *args, **kwargs)
---> 25 return await loop.run_in_executor(None, func_call)

File /usr/lib/python3.10/concurrent/futures/thread.py:58, in _WorkItem.run(self
     55     return
     57 try:
---> 58     result = self.fn(*self.args, **self.kwargs)
     59 except BaseException as exc:
     60     self.future.set_exception(exc)

Cell In[27], line 19, in SketchNegotiator.propose_sketch(self, problem)
     17 # Check cache first
     18 cache_key = f"sketch:{hash(problem)}"
---> 19 cached = cache.get(cache_key)
     20 if cached:
     21     print("Using cached sketch")
```

```
File ~/.local/lib/python3.10/site-packages/redis/commands/core.py:1829, in␣
 ↪BasicKeyCommands.get(self, name)
   1823 def get(self, name: KeyT) -> ResponseT:
   1824     """
   1825     Return the value at key ``name``, or None if the key doesn't exist
   1826
   1827     For more information see https://redis.io/commands/get
   1828     """
-> 1829     return self.execute_command("GET", name, keys=[name])

File ~/.local/lib/python3.10/site-packages/redis/client.py:623, in Redis.
 ↪execute_command(self, *args, **options)
    622 def execute_command(self, *args, **options):
--> 623     return self._execute_command(*args, **options)

File ~/.local/lib/python3.10/site-packages/redis/client.py:629, in Redis.
 ↪_execute_command(self, *args, **options)
    627 pool = self.connection_pool
    628 command_name = args[0]
--> 629 conn = self.connection or pool.get_connection()
    631 if self._single_connection_client:
    632     self.single_connection_lock.acquire()

File ~/.local/lib/python3.10/site-packages/redis/utils.py:191, in␣
 ↪deprecated_args.<locals>.decorator.<locals>.wrapper(*args, **kwargs)
    186     elif arg in provided_args:
    187         warn_deprecated_arg_usage(
    188             arg, func.__name__, reason, version, stacklevel=3
    189         )
--> 191 return func(*args, **kwargs)

File ~/.local/lib/python3.10/site-packages/redis/connection.py:1530, in␣
 ↪ConnectionPool.get_connection(self, command_name, *keys, **options)
   1526     self._in_use_connections.add(connection)
   1528 try:
   1529     # ensure this connection is connected to Redis
-> 1530     connection.connect()
   1531     # connections that the pool provides should be ready to send
   1532     # a command. if not, the connection was either returned to the
   1533     # pool before all data has been read or the socket has been
   1534     # closed. either way, reconnect and verify everything is good.
   1535     try:

File ~/.local/lib/python3.10/site-packages/redis/connection.py:379, in␣
 ↪AbstractConnection.connect(self)
    377 def connect(self):
    378     "Connects to the Redis server if not already connected"
--> 379     self.connect_check_health(check_health=True)
```

```
File ~/.local/lib/python3.10/site-packages/redis/connection.py:391, in␣
 ↪AbstractConnection.connect_check_health(self, check_health)
    389     raise TimeoutError("Timeout connecting to server")
    390 except OSError as e:
--> 391     raise ConnectionError(self._error_message(e))
    393 self._sock = sock
    394 try:

ConnectionError: Error 111 connecting to localhost:6379. Connection refused.
```

[ ]:

[ ]:

[ ]: